

Kubernetes Dashboard Access from Local Machine

RBAC + Kind + AWS EC2 + kubectl proxy + SSH Tunnel



Created By: Sagar Pyakurel

This project is made for educational purpose to guide anyone who wants to understand Kubernetes from beginner to advance in a short time through the step by step hands-on guide.

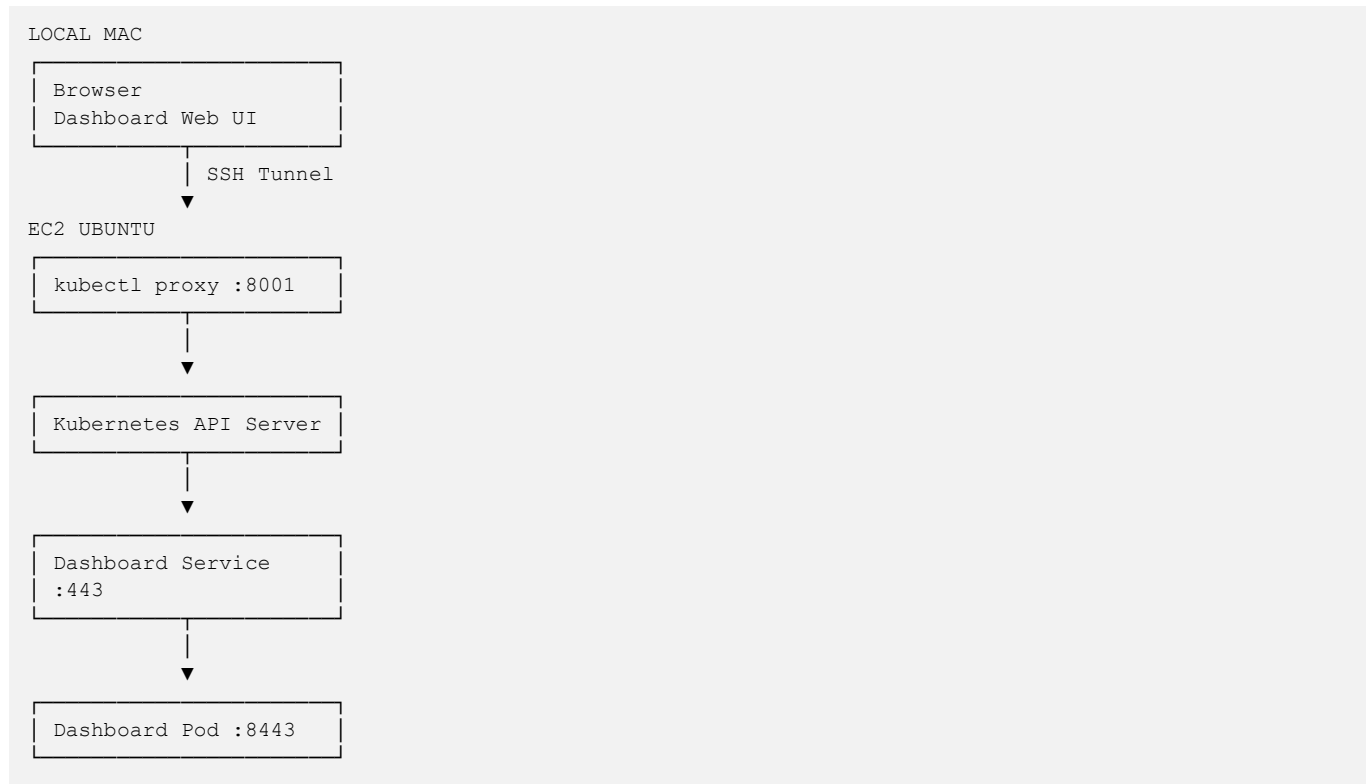
This documentation specially focus on the process of ClusterRoleBinding meaning is service(user) binding to any clusterRole. This is done in cluster level.

Current status note: Kubernetes Dashboard was archived and is no longer maintained.

Project Overview

This project demonstrates how to access the Kubernetes Dashboard running inside a Kind Kubernetes cluster on an AWS EC2 Ubuntu server from a local Mac browser. It also demonstrates Kubernetes Role-Based Access Control (RBAC) by creating a custom ServiceAccount and connecting it to the cluster-admin ClusterRole with a ClusterRoleBinding.

Architecture



Main concepts:

- Kubernetes Dashboard and Dashboard Service
- ServiceAccount, ClusterRole, and ClusterRoleBinding
- ServiceAccount token authentication
- Kubernetes API Server and RBAC authorization
- kubectl proxy for API access
- SSH local port forwarding from Mac to EC2

1. Environment

The Kubernetes cluster runs on an AWS EC2 Ubuntu server using Kind. The example cluster is named my-cluster and uses Kubernetes v1.31.2.

Run on EC2: (ubuntu instance in my case)

```
ssh -i "public_key.pem" ubuntu@YOUR_EC2_PUBLIC_IP
```

```
mkdir Dashboard && cd Dashboard
```

```
sudo apt install -y kubectx (this will dow kubectx and kubens)
```

kind-config.yaml

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
```

```
nodes:
```

- role: control-plane
image: kindest/node:v1.31.2
- role: worker
image: kindest/node:v1.31.2
- role: worker
image: kindest/node:v1.31.2
- role: worker
image: kindest/node:v1.31.2
extraPortMappings:
 - containerPort: 80
hostPort: 80
protocol: TCP
- containerPort: 443
hostPort: 443
protocol: TCP

```
Kubectl create cluster --name my-cluster --config config.yaml
```

```
kubectl cluster-info
```

```
kubectl get nodes
```

```
kubectl config current-context
```

```
kubectl apply -f
```

```
https://raw.githubusercontent.com/kubernetes/dashboard/v2.7.0/aio/deploy/recommended.yaml
```

```
kubens (it like kubectl get namespace)
```

```
kubens kubernetes-dashboard (This Activates kubernetes-dashboard namespace)
```

Expected: the Kind nodes are Ready and the current context is typically kind-my-cluster.

2. Verify Kubernetes Dashboard

Run on EC2:

```
kubectl get namespace kubernetes-dashboard
kubectl get pods -n kubernetes-dashboard
kubectl get svc -n kubernetes-dashboard
kubectl get endpoints -n kubernetes-dashboard
```

The important checks are:

- Dashboard Pods show Running.
- The Dashboard Service exposes port 443/TCP.
- The Service has a non-empty endpoint pointing to the Dashboard Pod (typically port 8443).

3. Create the RBAC ServiceAccount

Create **service_account.yaml** Run the file creation and apply commands on EC2.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: admin-user
  namespace: kubernetes-dashboard

kubectl apply -f service_account.yaml
kubectl get serviceaccount -n kubernetes-dashboard
```

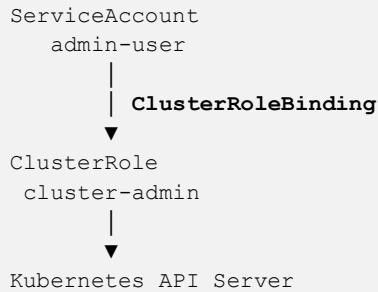
4. Create the ClusterRoleBinding

Create **cluster_role_binding.yaml** This connects the admin-user ServiceAccount to the cluster-admin ClusterRole.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: admin-user-binding
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
subjects:
- kind: ServiceAccount
  name: admin-user
  namespace: kubernetes-dashboard

kubectl apply -f cluster_role_binding.yaml
kubectl describe clusterrolebinding admin-user-binding
```

RBAC relationship



5. Verify RBAC Permissions

Run on EC2:

```
kubectl auth can-i get pods \
  --as=system:serviceaccount:kubernetes-dashboard:admin-user

kubectl auth can-i get nodes \
  --as=system:serviceaccount:kubernetes-dashboard:admin-user
```

Expected result for the cluster-admin binding: yes

```
kubectl auth can-i --list \
  --as=system:serviceaccount:kubernetes-dashboard:admin-user
```

6. Generate the Dashboard Token

Run on EC2:

```
kubectl -n kubernetes-dashboard create token admin-user
```

(-n kubernetes-dashboard is optional for use since we activated this namespace using kubens)

Copy the complete token. You will paste it into the Dashboard login screen. Modern Kubernetes uses short-lived ServiceAccount tokens rather than automatically creating a long-lived token Secret.

7. Start kubectl proxy

Run on EC2 in Terminal 1:

```
kubectl proxy --port=8001
```

do not forget to open the port in ec2 from security Group inbound rule

Keep this terminal running. kubectl proxy listens on EC2 localhost:8001 and provides access to the Kubernetes API Server.

8. Test kubectl proxy

Open another EC2 terminal (Terminal 2).

```
curl http://127.0.0.1:8001/version
curl http://127.0.0.1:8001/api/
```

The /version response should show your Kubernetes server version, for example v1.31.2. The /api/ response should include API version v1.

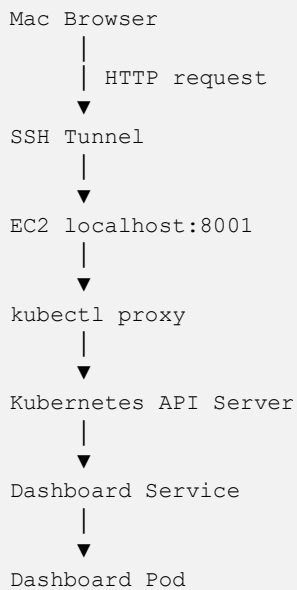
9. Dashboard API Proxy URL

Your Dashboard namespace and Service are both named `kubernetes-dashboard`. The generic API proxy pattern is:

```
/api/v1/namespaces/<namespace>/services/<service>/proxy/
```

```
http://localhost:8001/api/v1/namespaces/kubernetes-dashboard/services/https:kubernetes-  
dashboard:/proxy/
```

kubectl proxy flow



10. Create the SSH Tunnel from the Local Mac

The kubectl proxy is bound to EC2 localhost, so your Mac cannot directly use that localhost. Create a local SSH tunnel from the Mac.

Run on LOCAL MAC:

```
ssh -i "k8s-in-one-shot.pem" \  
-L 9001:127.0.0.1:8001 \  
ubuntu@YOUR_EC2_PUBLIC_IP
```

Example used in the project: ubuntu@18.189.193.96. Port 9001 belongs to the Mac; port 8001 is the EC2 kubectl proxy port.

11. Test the SSH Tunnel

Open another LOCAL MAC terminal while the SSH session is still running.

```
curl http://127.0.0.1:9001/version
```

If the Mac receives the Kubernetes version JSON, the full Mac -> SSH -> EC2 -> kubectl proxy -> API Server path is working.

12. Open the Dashboard in the Local Browser

Open this URL on the LOCAL MAC browser:

```
http://localhost:9001/api/v1/namespaces/kubernetes-dashboard/services/https:kubernetes-  
dashboard:/proxy/
```

Use http:// for the local kubectl proxy endpoint. Do not change it to https://localhost:9001 for this proxy-based flow.

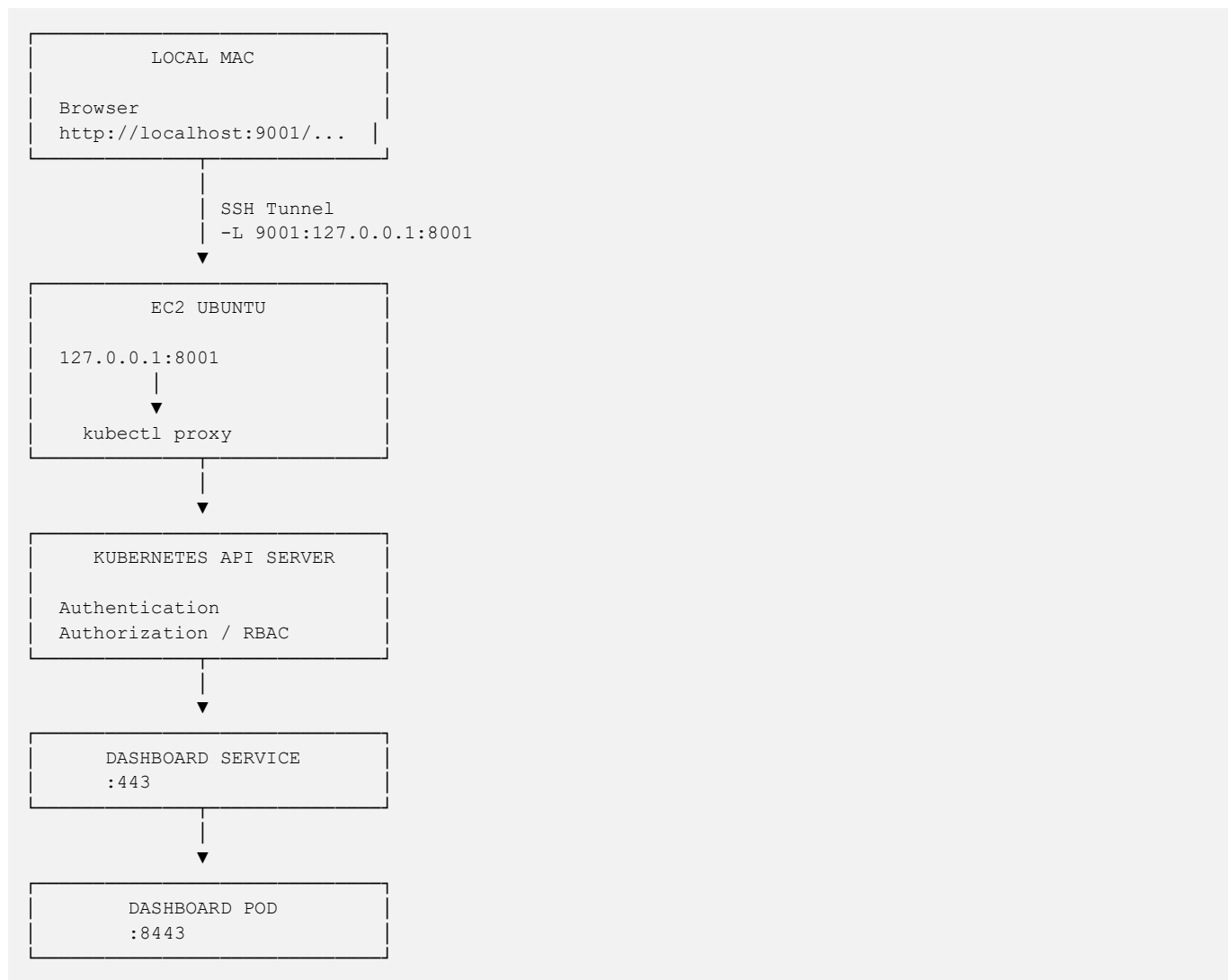
13. Log In to the Dashboard

On the Dashboard login screen:

- Select Token.
- Paste the token generated on EC2 with create token admin-user.
- Sign in.

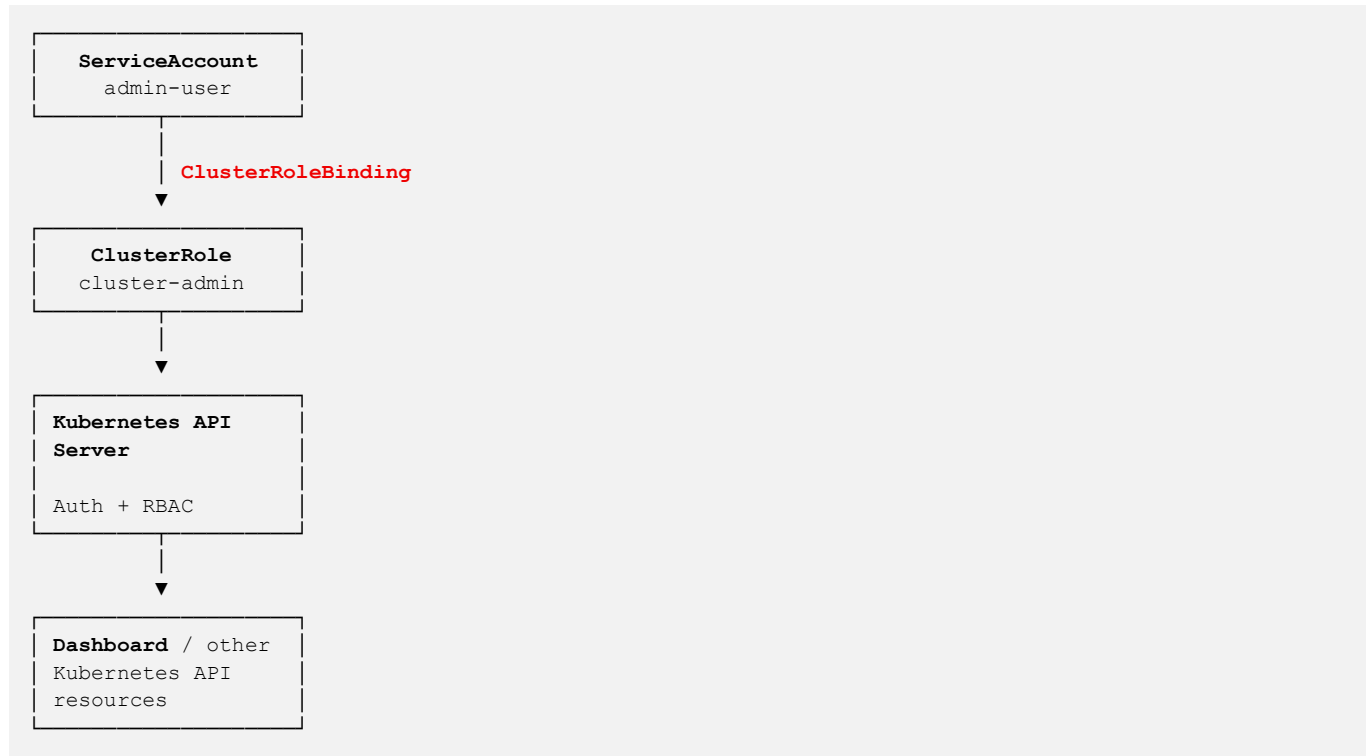
14. Complete End-to-End Flow

End-to-end architecture



15. RBAC Flow

Authorization flow



16. Important Ports

6443 → Kubernetes API Server
8001 → kubectl proxy on EC2
9001 → SSH tunnel port on Mac
443 → Dashboard Service
8443 → Dashboard Pod

17. Terminal Setup

EC2 Terminal 1:

```
kubectl proxy --port=8001
```

Mac Terminal 1:

```
ssh -i "k8s-in-one-shot.pem" \  
-L 9001:127.0.0.1:8001 \  
ubuntu@YOUR_EC2_PUBLIC_IP
```

Mac Browser:

```
http://localhost:9001/api/v1/namespaces/kubernetes-dashboard/services/https:kubernetes-  
dashboard:/proxy/
```

18. Troubleshooting Commands

Dashboard health - EC2:

```
kubectl get pods -n kubernetes-dashboard
kubectl get svc -n kubernetes-dashboard
kubectl get endpoints -n kubernetes-dashboard
```

RBAC - EC2:

```
kubectl describe clusterrolebinding admin-user-binding
kubectl auth can-i get pods --as=system:serviceaccount:kubernetes-dashboard:admin-user
```

kubectl proxy - EC2:

```
ss -lntp | grep 8001
curl http://127.0.0.1:8001/version
```

SSH tunnel - Mac:

```
curl http://127.0.0.1:9001/version
```

19. What This Project Demonstrates

- Kind Kubernetes cluster running on AWS EC2
- Kubernetes Dashboard installation and verification
- ServiceAccount creation
- ClusterRoleBinding and RBAC authorization
- ServiceAccount token authentication
- Kubernetes API Server access
- kubectl proxy
- Kubernetes Service discovery
- SSH local port forwarding
- Remote EC2-to-local-browser access

20. Main Learning Summary

Concept summary

RBAC

- | ServiceAccount → WHO?
- | ClusterRole → WHAT permissions?
- | ClusterRoleBinding → WHO gets them?

Networking

- Service → stable application endpoint
- kubectl proxy → local API proxy
- SSH tunnel → local Mac access to EC2 localhost

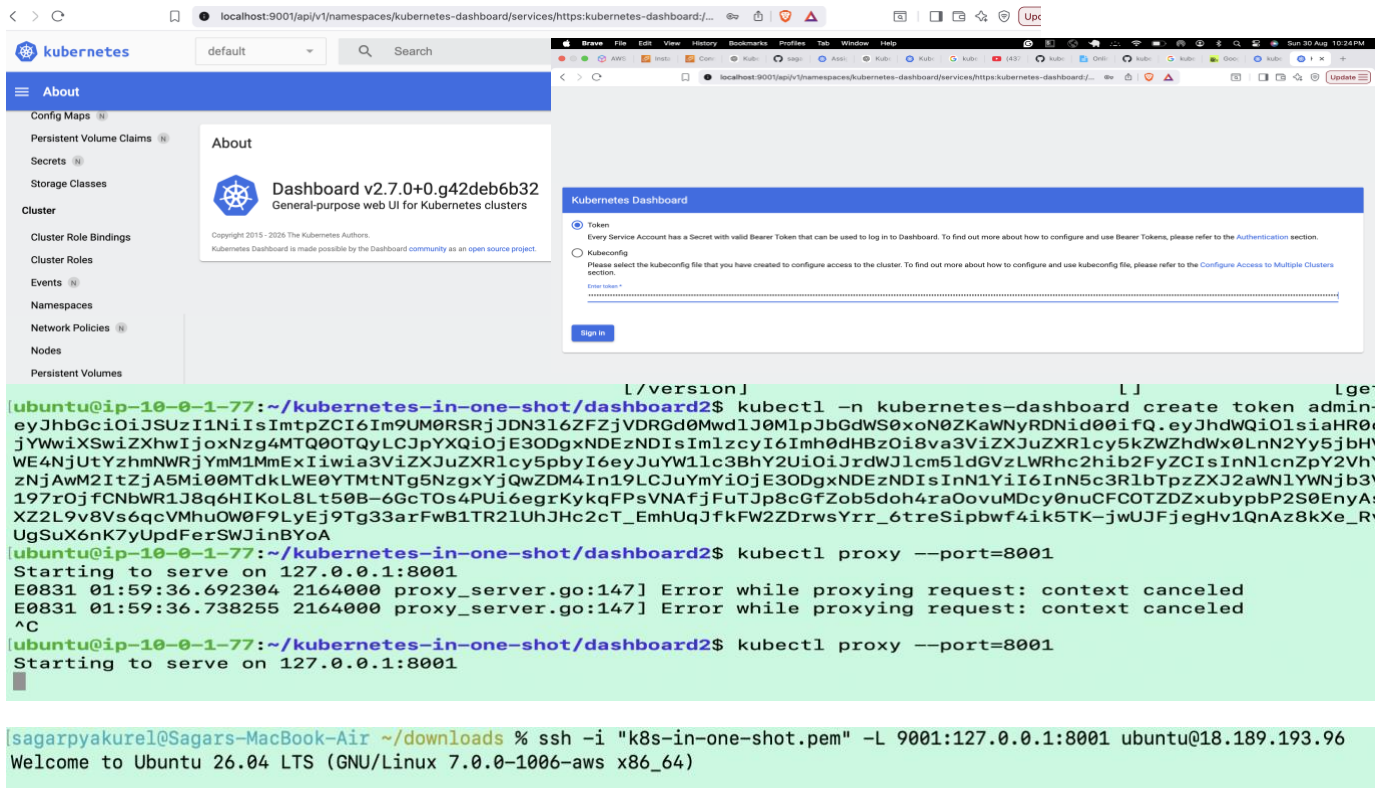
Final path

Mac Browser → SSH → EC2 → kubectl proxy → API Server → Dashboard Service → Dashboard Pod

21. GitHub Project Description

Implemented Kubernetes Dashboard access on a Kind cluster running on AWS EC2 using RBAC with a custom ServiceAccount and ClusterRoleBinding, authenticated with a ServiceAccount token, exposed through kubectl proxy, and securely accessed from a local Mac browser using SSH port forwarding.

22. Appendix



23. Official References

- **Kubernetes Dashboard repository (archived):** <https://github.com/kubernetes-retired/dashboard>
- **Kubernetes Dashboard documentation:** <https://kubernetes.io/docs/tasks/access-application-cluster/web-ui-dashboard/>
- **kubectl proxy reference:** https://kubernetes.io/docs/reference/kubectl/generated/kubectl_proxy/
- **kubectl port-forward reference:** https://kubernetes.io/docs/reference/kubectl/generated/kubectl_port-forward/